

# Части II и III — Как да напишем `class.cpp`

ръководство стъпка по стъпка

Microsoft Visual Studio · C++17 · от структура към клас

## Какво ще направим

Ще преработим структурата от част I в **клас**. Три неща се променят и всяко от тях решава конкретен проблем:

| Промяна                                   | Проблемът в част I                                                            | Какво решава                                                    |
|-------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <code>struct</code> → <code>class</code>  | Всеки може да пише директно в <code>m_szName</code> и да заобиколи проверките | Данните стават <b>приватни</b> — достъпът е само през функциите |
| <code>char[]</code> → <code>string</code> | Ограничена дължина, ръчно копиране, риск от препълване                        | <code>string</code> управлява паметта сам                       |
| текст → <code>enum</code>                 | В полето за пол може да се запише какъвто и да е текст                        | Само три допустими стойности, проверени при компилация          |

Добавяме и **ЕГН** заедно с функция, която проверява дали е валидно по алгоритъма на ГРАО.

**План на работата**

- Нов проект и скелет
- Изброим тип `Gender` (изискване II)
- Класът и член-променливите (изискване III.1)
- Функции за четене и презапис (изискване III.3)
- Преобразуване на изброимия тип (изискване III.2)
- Валидизатор на ЕГН (изискване III.3)
- Конструктори (изискване III.3)
- Главна функция с тестове (изискване III.5)

## 1 Нов проект и скелет

**Направете НОВ проект**, не добавяйте файла към проекта от част I. Двата файла съдържат по една функция `main` и Visual Studio ще откаже с грешка `LNK2005: main already defined`.

Ако все пак искате двата файла в един проект: щракнете с десен бутон върху единия → **Properties** → **Excluded From Build: Yes**. Така той се пропуска при компилация.

СЪЗДАЙТЕ `CLASS.CPP` И НАПИШЕТЕ

```
#include <cctype>      // isdigit
#include <cstdio>      // getchar
#include <iostream>    // cout
#include <string>      // string
using namespace std;

int main()
{
    cout << "Програмата работи!" << endl;

    getchar();          // задържа прозореца отворен при F5
    return 0;
}
```

Тук **не** е нужен `#define _CRT_SECURE_NO_WARNINGS` както в част I. Той беше необходим само заради `strncpy`. Класът използва `string`, който Visual Studio не смята за опасен — едно от практическите предимства на STL.

**Проверка:** `Ctrl + Shift + B` трябва да даде 1 succeeded, 0 failed.

## 2 Изброим тип `Gender` (изискване II)

Изброимият тип е нов тип данни, който има **краен списък от допустими стойности**. Вместо полът да е текст, в който може да се напише каквото и да е, той става тип с точно три възможности.

### ДОБАВЕТЕ НАД `MAIN`

```
enum Gender
{
    MALE,          // мъж
    FEMALE,        // жена
    UNKNOWN        // неопределен
};
```

### ОБЯСНЕНИЕ

Компилаторът присвоява числа автоматично: `MALE` е 0, `FEMALE` е 1, `UNKNOWN` е 2. Затова, ако изведете стойността директно с `cout`, ще видите число, а не текст — точно затова в стъпка 5 пишем функция за преобразуване.

**Каква е ползата?** При `char m_szGender[32]` нищо не пречи да се запише "хеликоптер" — грешката ще се открие едва когато някой прочете данните. При `Gender m_gender` присвояването на нещо извън списъка **не се компилира**. Грешката се хваща веднага, а не по време на изпълнение.

Не пропускайте **точката и запетая** след затварящата скоба: `};`

## 3 Класът и член-променливите (изискване III.1)

### ДОБАВЕТЕ ПОД `ENUM GENDER`

```
class CPerson2
{
private:
    string m_strName;    // име
    Gender m_gender;     // пол
    string m_strEGN;     // ЕГН

public:
    // тук ще добавим функциите
};
```

#### РАЗЛИКАТА МЕЖДУ STRUCT И CLASS

Технически тя е **една-единствена**: в `struct` всичко е публично по подразбиране, а в `class` — приватно. Всичко останало работи еднакво.

| Спецификатор          | Кой има достъп                                                          |
|-----------------------|-------------------------------------------------------------------------|
| <code>private:</code> | Само функциите на самия клас. Отвън достъпът е забранен от компилатора. |
| <code>public:</code>  | Всеки. Тук се обявява това, което класът предлага на външния свят.      |

Това е **капсулация** — една от основните идеи на ООП. Данните са скрити, а достъпът минава задължително през функции, в които може да се сложи проверка. Точно това ще използваме при ЕГН: невалидна стойност няма да бъде записана.

**Проверете сами:** добавете временно в `main` реда `CPerson2 p; p.m_strName = "X";` и компилирайте. Visual Studio отказва с C2248: '`CPerson2::m_strName`': cannot access private member. Същият ред в част I би се компилирал без проблем.

## 4 Функции за четене и презапис (изискване III.3)

Ще ги обявим в класа, а ще ги напишем след него. Така тялото на класа остава кратък списък на това, което той предлага.

#### ДОБАВЕТЕ СЛЕД PUBLIC:

```
void SetName(const string& strName);
void SetGender(Gender gender);
void SetGender(const string& strGender);    // предефинирана
bool SetEGN(const string& strEGN);

const string& GetName() const;
Gender GetGender() const;
const string& GetEGN() const;
```

#### ДОБАВЕТЕ СЛЕД ЗАТВАРЯЩАТА СКОБА НА КЛАСА

```

void CPerson2::SetName(const string& strName)
{
    m_strName = strName;
}

const string& CPerson2::GetName() const
{
    return m_strName;
}

Gender CPerson2::GetGender() const
{
    return m_gender;
}

const string& CPerson2::GetEGN() const
{
    return m_strEGN;
}

```

### ТРИ НЕЩА ЗА ОБЯСНЕНИЕ

| Запис                         | Значение                                                                                                                                                                  |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CPerson2::                    | Операторът за обхват. Извън тялото на класа компилаторът трябва да знае на кой клас принадлежи функцията. Ако го пропуснете, ще получите грешка за неизвестни променливи. |
| const string& като параметър  | Предаване по <b>референция</b> — не се прави копие на текста. Думата const гарантира, че функцията няма да го промени.                                                    |
| const string& като върнат тип | Връща се препратка към вътрешния низ, отново без копиране. const пречи на промяна отвън.                                                                                  |

### ДВЕТЕ ВЕРСИИ НА SETGENDER

```

void CPerson2::SetGender(Gender gender)
{
    m_gender = gender;
}

void CPerson2::SetGender(const string& strGender)
{
    m_gender = stringToGender(strGender);    // ще я напишем в стъпка 5
}

```

Две функции с едно и също име, но с различни параметри, се наричат **предефинирани** (overloaded). Компилаторът избира правилната според типа на аргумента: SetGender(MALE) вика първата, SetGender("мъж") — втората.

**Проверка:** в този момент кодът *няма* да се компилира, защото stringToGender още не съществува. Продължете със стъпка 5.

## 5 Преобразуване на изброимия тип (изискване III.2)

Нужни са ни две функции в противоположни посоки: от текст към Gender и обратно.

## ОБЯВЕТЕ В КЛАСА, СЛЕД ОСТАНАЛИТЕ ФУНКЦИИ

```
Gender stringToGender(const string& strGender) const;
string genderToString() const;
```

## НАПИШЕТЕ СЛЕД КЛАСА

```
Gender CPerson2::stringToGender(const string& strGender) const
{
    if (strGender == "мъж" || strGender == "м" || strGender == "male")
        return MALE;
    if (strGender == "жена" || strGender == "ж" || strGender == "female")
        return FEMALE;
    return UNKNOWN;
}

string CPerson2::genderToString() const
{
    switch (m_gender)
    {
        case MALE:    return "мъж";
        case FEMALE:  return "жена";
        default:      return "неопределен";
    }
}
```

## ЗАЩО СА РАЗЛИЧНИ ПО УСТРОЙСТВО

Забележете асиметрията: `stringToGender` получава текст като **параметър**, докато `genderToString` не приема нищо — тя чете `m_gender` на самия обект. Така са зададени в условието.

Сравнението `strGender == "мъж"` работи, защото `string` има вграден оператор за сравнение. При масивите от символи в част I това щеше да изисква `strcmp`.

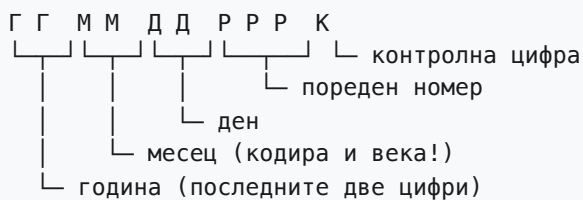
Непознат текст дава `UNKNOWN` вместо грешка — съзнателно решение, което прави функцията безопасна за всякакъв вход.

В `switch` клаузата `default` покрива `UNKNOWN`. Ако я пропуснете, компилаторът предупреждава, че не всички пътища връщат стойност.

## 6 Валидизатор на ЕГН (изискване III.3)

Това е най-обемната функция. Ще я построим на **три части** и ще проверяваме след всяка.

ЕГН се състои от 10 цифри със следното значение:



## ОБЯВЕТЕ В КЛАСА

```
bool isValid(const string& strEGN);
```

## 6.1 Проверка на формата

## НАПИШЕТЕ СЛЕД КЛАСА

```
bool CPerson2::isValid(const string& strEGN)
{
    // a) точно 10 символів, всі цифри
    if (strEGN.length() != 10)
        return false;
    for (size_t i = 0; i < strEGN.length(); i++)
        if (!isdigit(static_cast<unsigned char>(strEGN[i])))
            return false;

    return true;    // временно — ще заміним в 6.2
}
```

Записът `static_cast<unsigned char>` изглежда излишен, но е нужен: `isdigit` се обърква при символи с код над 127, каквито са байтовете на кирилицата. Преобразуването гарантира правилна работа.

Типът `size_t` се използва за броячи по дължина на низ. Ако напишете `int`, компилаторът дава предупреждение за сравнение между знаков и беззнаков тип.

**Проверка:** компилирайте. В `main` напишете временно:

```
CPerson2 t; cout << t.isValid("1234567890") << t.isValid("a6v") << endl;
```

Очакван изход: 10 — първото минава проверката за формат, второто не.

## 6.2 Проверка на датата

Месецът носи информация и за **века**:

| Стойност на месеца | Век         | Действителен месец  |
|--------------------|-------------|---------------------|
| 21 – 32            | 1800 – 1899 | стойността минус 20 |
| 01 – 12            | 1900 – 1999 | самата стойност     |
| 41 – 52            | 2000 – 2099 | стойността минус 40 |

ЗАМЕНЕТЕ `RETURN TRUE;` С ТОЗИ КОД

```
// 6) валидна дата на раждане
int nYear = (strEGN[0] - '0') * 10 + (strEGN[1] - '0');
int nMonth = (strEGN[2] - '0') * 10 + (strEGN[3] - '0');
int nDay = (strEGN[4] - '0') * 10 + (strEGN[5] - '0');

if (nMonth >= 1 && nMonth <= 12)
    nYear += 1900;
else if (nMonth >= 21 && nMonth <= 32)
{   nYear += 1800; nMonth -= 20;   }
else if (nMonth >= 41 && nMonth <= 52)
{   nYear += 2000; nMonth -= 40;   }
else
    return false; // несъществуващ месец

int arrDays[12] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
bool bLeap = (nYear % 4 == 0 && nYear % 100 != 0) || (nYear % 400 == 0);
if (nMonth == 2 && bLeap)
    arrDays[1] = 29;
if (nDay < 1 || nDay > arrDays[nMonth - 1])
    return false; // несъществуващ ден

return true; // временно – ще заменим в 6.3
```

#### ДВАТА ТРИКА В ТОЗИ КОД

**Изваждането на '0'.** Символът '7' се пази в паметта като число 55, а '0' — като 48. Разликата 55 – 48 = 7 дава цифрата. Затова `strEGN[0] - '0'` превръща символ в число.

**Правилото за високосна година.** Условието `(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)` не е излишно усложнение. Годините, кратни на 100, не са високосни, освен ако не се делят и на 400. Затова 1900 не е високосна, а 2000 — е.

**Проверка — тук се вижда защо правилото е важно.** Тествайте тези две стойности:

| ЕГН        | Резултат  | Защо                                                                                   |
|------------|-----------|----------------------------------------------------------------------------------------|
| 0042290000 | валидно   | месец 42 → февруари <b>2000</b> г.; 2000 се дели на 400 → високосна                    |
| 0002290000 | невалидно | месец 02 → февруари <b>1900</b> г.; 1900 се дели на 100, но не на 400 → не е високосна |

Двете се различават само по месеца, а резултатът е противоположен.

### 6.3 Контролна цифра

Всяка от първите девет цифри се умножава по свое тегло:

| Позиция | 1 | 2 | 3 | 4 | 5  | 6 | 7 | 8 | 9 |
|---------|---|---|---|---|----|---|---|---|---|
| Тегло   | 2 | 4 | 8 | 5 | 10 | 9 | 7 | 3 | 6 |

Сумата се дели на 11 с остатък. Ако остатъкът е 10, той се заменя с 0. Резултатът трябва да съвпадне с десетата цифра.

**ЗАМЕНЕТЕ ПОСЛЕДНОТО RETURN TRUE;**

```

// в) контролна цифра
const int arrWeights[9] = { 2, 4, 8, 5, 10, 9, 7, 3, 6 };
int nSum = 0;
for (int i = 0; i < 9; i++)
    nSum += (strEGN[i] - '0') * arrWeights[i];

int nCheck = nSum % 11;
if (nCheck == 10)
    nCheck = 0;

return nCheck == (strEGN[9] - '0');
}

```

#### ПРОВЕРКА НА ПРИМЕРИТЕ ОТ УСЛОВИЕТО

**ЕГН 111111110** — първите девет цифри са единици, затова сумата е равна на сбора на теглата:

```

2+4+8+5+10+9+7+3+6 = 54
54 mod 11 = 10 → заменя се с 0
контролна цифра = 0 ✓ съвпада с последната цифра

```

**ЕГН 111111104** — деветата цифра е 0, затова последното тегло отпада:

```

54 - 6 = 48
48 mod 11 = 4
контролна цифра = 4 ✓ съвпада с последната цифра

```

**Проверка:** и двата примера от условието трябва да излязат валидни. Променете последната цифра на кой да е от тях — резултатът трябва да стане невалиден.

**Допълнение:** в реалния стандарт деветата цифра кодира и пола — *четна* означава мъж, *нечетна* — жена. Условието не го изисква, но е възможно разширение: сравнение между пола от ЕГН и стойността на `m_gender`.

## 6.4 Използване на валидизатора

Сега `SetEGN` може да пази данните на обекта от невалидни стойности.

#### НАПИШЕТЕ СЛЕД КЛАСА

```

bool CPerson2::SetEGN(const string& strEGN)
{
    if (!IsValid(strEGN))
        return false;           // невалидно => старата стойност се запазва

    m_strEGN = strEGN;
    return true;
}

```

Ето ползата от капсулацията: понеже `m_strEGN` е приватна, **няма начин** невалидно ЕГН да попадне в обекта. Ако променливата беше публична, всеки можеше да я презапише директно.

## 7 Конструктори (изискване III.3)

ОБЯВЕТЕ В КЛАСА, В НАЧАЛОТО НА СЕКЦИЯТА `PUBLIC:`

```
CPerson2();  
CPerson2(const string& strName, const string& strGender, const string& strEGN);
```

НАПИШЕТЕ СЛЕД КЛАСА

```
CPerson2::CPerson2()  
    : m_strName("Неизвестен"), m_gender(UNKNOWN), m_strEGN("")  
{  
}  
  
CPerson2::CPerson2(const string& strName, const string& strGender, const string& strEGN)  
    : m_strName(strName), m_gender(UNKNOWN), m_strEGN("")  
{  
    m_gender = stringToGender(strGender);  
    SetEGN(strEGN);  
}
```

КАКВО Е ДВОЕТОЧИЕТО СЛЕД ИМЕТО

Това е **списък за инициализация**. Стойностите се задават направо при създаването на променливите, вместо да се присвояват след това в тялото. За `string` разликата е реална: без списъка низът първо се създава празен и веднага след това се презаписва — излишна работа.

В третия ред на втория конструктор ЕГН минава през `SetEGN`, а не се присвоява направо. Така **и при създаване** невалидна стойност бива отхвърлена и обектът остава с празно ЕГН.

**Проверка:** компилирайте и добавете в `main`:

```
CPerson2 a; CPerson2 b("Иван Петров", "мъж", "1111111110");  
Изведете имената им — първото трябва да е „Неизвестен“.
```

Функция за извеждане

ОБЯВЕТЕ В КЛАСА `VOID PRINT() CONST;` И НАПИШЕТЕ СЛЕД НЕГО

```
void CPerson2::Print() const  
{  
    cout << GetName() << " (" << genderToString() << "), ЕГН: "  
        << (GetEGN().empty() ? string("(няма)") : GetEGN()) << endl;  
}
```

Записът `условие ? а : б` е съкратен `if`: ако ЕГН е празно, извежда „(няма)“, иначе — самото ЕГН.

## 8 Главна функция с тестове (изискване III.5)

Условието иска обекти от различни видове и проверка на всички функции. Добавяйте по една секция и компилирайте след всяка.

## 8.1 Двата конструктора

```
cout << "---- Обикновени обекти ----" << endl;
CPerson2 person1;                                // подразбиращ се
CPerson2 person2("Иван Петров", "мъж", "111111110"); // експлицитен
person1.Print();
person2.Print();
```

## 8.2 Функции за презапис

```
cout << "---- След промяна ----" << endl;
person1.SetName("Мария Иванова");
person1.SetGender(FEMALE);           // чрез изброимия тип
person1.SetEGN("1111111104");
person1.Print();
```

Тук `SetGender` получава `FEMALE` — избира се версията с параметър `Gender`. В 8.5 ще подадем текст и ще се избере другата.

## 8.3 Масив от обекти

```
cout << "---- Масив от обекти ----" << endl;
CPerson2 people[3] = {
    CPerson2("Георги Стоянов", "м", "111111110"),
    CPerson2("Елена Димитрова", "ж", "1111111104"),
    CPerson2()
};
const int nCount = sizeof(people) / sizeof(people[0]);
for (int i = 0; i < nCount; i++)
    people[i].Print();
```

## 8.4 Указател и динамична памет

```
cout << "---- Указател (динамична памет) ----" << endl;
CPerson2* pPerson = new CPerson2("Петър Колев", "мъж", "111111110");
pPerson->Print();
delete pPerson;
pPerson = nullptr;
```

## 8.5 Референция

```
cout << "---- Референция ----" << endl;
CPerson2& refPerson = person2;
refPerson.SetGender("жена");           // текст => другата версия
person2.Print();
```

Референцията не е копие. Промяната през `refPerson` се вижда при отпечатване на `person2`.

## 8.6 Константен обект

```
cout << "---- Константен обект ----" << endl;
const CPerson2 cPerson("Анна Тодорова", "жена", "1111111104");
cout << cPerson.GetName() << " / " << cPerson.genderToString()
    << " / " << cPerson.GetEGN() << endl;
// cPerson.SetName("Друго");           // грешка: обектът е константен
```

**Задача:** разкоментирайте последния ред. Компилацията трябва да **откаже** с C2662 . Това доказва защо функциите за четене са маркирани с `const` — без тази дума дори четенето от константен обект би било невъзможно.

## 8.7 Тест на преобразуването

```
cout << "---- Тест: текст -> Gender -> текст ----" << endl;
const char* arrTests[] = { "мъж", "жена", "М", "Ж", "male", "female", "нещо" };
CPerson2 tester;
for (size_t i = 0; i < sizeof(arrTests) / sizeof(arrTests[0]); i++)
{
    tester.SetGender(string(arrTests[i]));
    cout << "  \" << arrTests[i] << "\" -> " << tester.GetGender()
        << " -> \"" << tester.genderToString() << "\"\" << endl;
}
```

В изхода се виждат числата 0, 1 и 2 — вътрешните стойности на изброимия тип. Записът `string(arrTests[i])` е нужен, за да се избере версията на `SetGender` с текстов параметър.

## 8.8 Тест на валидизатора

```
cout << "---- Тест: валидност на ЕГН ----" << endl;
const char* arrEGN[] = {
    "1111111110", // валидно (пример от условието)
    "1111111104", // валидно (пример от условието)
    "1111111111", // грешна контролна цифра
    "111111111",  // по-малко от 10 цифри
    "111111111a", // съдържа буква
    "1199111110", // несъществуващ месец
    "1111321110"  // несъществуващ ден
};
CPerson2 validator;
for (size_t i = 0; i < sizeof(arrEGN) / sizeof(arrEGN[0]); i++)
    cout << "  " << arrEGN[i] << " -> "
        << (validator.isValid(arrEGN[i]) ? "валидно" : "НЕвалидно") << endl;
```

Всеки ред проверява различно правило от алгоритъма — така се вижда, че функцията отхвърля по правилната причина, а не случайно.

## 8.9 Защита на данните

```
cout << "---- Тест: SetEGN пази старата стойност ----" << endl;
CPerson2 person3("Стефан Николов", "мъж", "1111111110");
cout << "  преди:  " << person3.GetEGN() << endl;
bool bResult = person3.SetEGN("1234567890"); // невалидно
cout << "  SetEGN върна: " << (bResult ? "true" : "false") << endl;
cout << "  след:   " << person3.GetEGN() << endl;

getchar();
return 0;
}
```

Това е най-важният тест: невалидната стойност е отхвърлена и старата е запазена.

## Компилиране и стартиране

### Вариант А — през средата на Visual Studio

| Клавиши          | Действие                                                                   |
|------------------|----------------------------------------------------------------------------|
| Ctrl + Shift + B | Build — само компилира.                                                    |
| F5               | Start Debugging — прозорецът се затваря веднага, затова е нужен getchar(). |
| Ctrl + F5        | Start Without Debugging — средата сама задържа прозореца.                  |

### Вариант Б — от команден ред

Компиляторът `cl` не работи в обикновен `cmd`. Отворете **Developer Command Prompt for VS** от старт менюто.

```
cl /EHsc /std:c++17 /utf-8 class.cpp
class.exe
```

| Флаг       | Значение                                                                                                        |
|------------|-----------------------------------------------------------------------------------------------------------------|
| /EHsc      | Включва обработката на изключения. STL типът <code>string</code> ги използва, затова тук флагът е задължителен. |
| /std:c++17 | Задава стандарта на езика.                                                                                      |
| /utf-8     | Указва, че файлът е записан в UTF-8. Задължителен при кирилица.                                                 |

**Кирилица в конзолата.** Ако виждате `???` вместо букви: запишете файла като **UTF-8 with signature** (File → Save As → стрелката до Save → Save with Encoding), компилирайте с `/utf-8` и изпълнете `chcp 65001` в конзолата преди програмата. Като алтернатива добавете `#include <cstdlib>` и като първи ред в `main` :

```
system("chcp 65001 > nul");
```

## Пълен код на програмата

```
// Тема 1: Въведение в ООП. Клас CPerson2 (STL string + изброим тип).
#include <cctype>      // isdigit
#include <cstdio>      // getchar
#include <iostream>
#include <string>
using namespace std;

// ===== II. Изброим тип за кодиране на пол =====
enum Gender
{
    MALE,        // мъж
    FEMALE,      // жена
    UNKNOWN      // неопределен
};

// ===== III. Клас CPerson2 =====
class CPerson2
{
// 1. struct -> class: членовете са ПРИВАТНИ по подразбиране,
//    затова достъпът до тях става само през функциите по-долу.
private:
    string m_strName;      // 1. масивът от символи -> STL тип string
    Gender m_gender;       // 1. полът -> изброим тип вместо масив от символи
    string m_strEGN;       // 1. нова променлива: ЕГН

// 3. Публична спецификация
public:
    // 3. Подразбиращ се конструктор
    CPerson2();

    // 3. Експлицитен конструктор
    CPerson2(const string& strName, const string& strGender, const string& strEGN);

    // 3. Функции за презапис (промяна) на член променливите
    void SetName(const string& strName);
    void SetGender(Gender gender);
    void SetGender(const string& strGender);    // предефинирана: приема текст
    bool SetEGN(const string& strEGN);         // записва само валидно ЕГН

    // 3. Функции за четене; const => не променят обекта
    const string& GetName() const;
    Gender GetGender() const;
    const string& GetEGN() const;

    // 2. Преобразуване между изброимия тип и читаем текст
    Gender stringToGender(const string& strGender) const;
    string genderToString() const;

    // 3. Валидизатор на ЕГН (алгоритъм на ГРА0)
    bool isValid(const string& strEGN);

    void Print() const;
};

// ----- 3. Конструктори -----
CPerson2::CPerson2()
```

```

        : m_strName("Неизвестен"), m_gender(UNKNOWN), m_strEGN("")
    {
    }

CPerson2::CPerson2(const string& strName, const string& strGender, const string& strEGN)
    : m_strName(strName), m_gender(UNKNOWN), m_strEGN("")
{
    m_gender = stringToGender(strGender);
    SetEGN(strEGN);
}

// ----- 3. Функции за презапис -----
void CPerson2::SetName(const string& strName)
{
    m_strName = strName;
}

void CPerson2::SetGender(Gender gender)
{
    m_gender = gender;
}

void CPerson2::SetGender(const string& strGender)
{
    m_gender = stringToGender(strGender);
}

bool CPerson2::SetEGN(const string& strEGN)
{
    if (!IsValid(strEGN))
        return false;          // невалидно ЕГН => старата стойност се запазва

    m_strEGN = strEGN;
    return true;
}

// ----- 3. Функции за четене -----
const string& CPerson2::GetName() const { return m_strName; }
Gender CPerson2::GetGender() const { return m_gender; }
const string& CPerson2::GetEGN() const { return m_strEGN; }

// ----- 2. Преобразуване на изброимия тип -----
Gender CPerson2::stringToGender(const string& strGender) const
{
    if (strGender == "мъж" || strGender == "М" || strGender == "male")
        return MALE;
    if (strGender == "жена" || strGender == "Ж" || strGender == "female")
        return FEMALE;
    return UNKNOWN;
}

string CPerson2::genderToString() const
{
    switch (m_gender)
    {
        case MALE:    return "мъж";
        case FEMALE:  return "жена";
        default:      return "неопределен";
    }
}

```

```

// ----- 3. Валидизатор на ЕГН -----
// ЕГН = ГГ ММ ДД РРР К (10 цифри)
// - месецът кодира и века: +20 => 18xx, +40 => 20xx
// - К е контролна цифра по тегла 2,4,8,5,10,9,7,3,6
bool CPerson2::isValid(const string& strEGN)
{
    // а) точно 10 символа, само цифри
    if (strEGN.length() != 10)
        return false;
    for (size_t i = 0; i < strEGN.length(); i++)
        if (!isdigit(static_cast<unsigned char>(strEGN[i])))
            return false;

    // б) валидна дата на раждане
    int nYear = (strEGN[0] - '0') * 10 + (strEGN[1] - '0');
    int nMonth = (strEGN[2] - '0') * 10 + (strEGN[3] - '0');
    int nDay = (strEGN[4] - '0') * 10 + (strEGN[5] - '0');

    if (nMonth >= 1 && nMonth <= 12)
        nYear += 1900;
    else if (nMonth >= 21 && nMonth <= 32)
    { nYear += 1800; nMonth -= 20; }
    else if (nMonth >= 41 && nMonth <= 52)
    { nYear += 2000; nMonth -= 40; }
    else
        return false; // несъществуващ месец

    int arrDays[12] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
    bool bLeap = (nYear % 4 == 0 && nYear % 100 != 0) || (nYear % 400 == 0);
    if (nMonth == 2 && bLeap)
        arrDays[1] = 29;
    if (nDay < 1 || nDay > arrDays[nMonth - 1])
        return false; // несъществуващ ден

    // в) контролна цифра
    const int arrWeights[9] = { 2, 4, 8, 5, 10, 9, 7, 3, 6 };
    int nSum = 0;
    for (int i = 0; i < 9; i++)
        nSum += (strEGN[i] - '0') * arrWeights[i];

    int nCheck = nSum % 11;
    if (nCheck == 10)
        nCheck = 0;

    return nCheck == (strEGN[9] - '0');
}

void CPerson2::Print() const
{
    cout << GetName() << " (" << genderToString() << "), ЕГН: "
        << (GetEGN().empty() ? string("няма") : GetEGN()) << endl;
}

// ===== 5. Главна функция: тестове =====
int main()
{
    // --- а) обикновени обекти: двата конструктора ---
    cout << "---- Обикновени обекти ----" << endl;
    CPerson2 person1; // подразбира се

```

```

CPerson2 person2("Иван Петров", "мъж", "111111110"); // експлицитен
person1.Print();
person2.Print();

// --- б) промяна чрез функциите за презапис ---
cout << "---- След промяна ----" << endl;
person1.SetName("Мария Иванова");
person1.SetGender(FEMALE); // чрез изброимия тип
person1.SetEGN("111111104");
person1.Print();

// --- в) масив от обекти ---
cout << "---- Масив от обекти ----" << endl;
CPerson2 people[3] = {
    CPerson2("Георги Стоянов", "м", "111111110"),
    CPerson2("Елена Димитрова", "ж", "111111104"),
    CPerson2()
};
const int nCount = sizeof(people) / sizeof(people[0]);
for (int i = 0; i < nCount; i++)
    people[i].Print();

// --- г) указател към обект в динамичната памет ---
cout << "---- Указател (динамична памет) ----" << endl;
CPerson2* pPerson = new CPerson2("Петър Колев", "мъж", "111111110");
pPerson->Print();
delete pPerson;
pPerson = nullptr;

// --- д) референция: псевдоним на съществуващ обект ---
cout << "---- Референция ----" << endl;
CPerson2& refPerson = person2;
refPerson.SetGender("жена"); // променя самия person2
person2.Print();

// --- е) константен обект: само const функции ---
cout << "---- Константен обект ----" << endl;
const CPerson2 cPerson("Анна Тодорова", "жена", "111111104");
cout << cPerson.GetName() << " / " << cPerson.genderToString()
    << " / " << cPerson.GetEGN() << endl;
// cPerson.SetName("Друго"); // грешка: обектът е константен

// --- ж) тест на stringToGender / genderToString ---
cout << "---- Тест: текст -> Gender -> текст ----" << endl;
const char* arrTests[] = { "мъж", "жена", "м", "ж", "male", "female", "нещо" };
CPerson2 tester;
for (size_t i = 0; i < sizeof(arrTests) / sizeof(arrTests[0]); i++)
{
    tester.SetGender(string(arrTests[i]));
    cout << "  \"\"\" << arrTests[i] << "\" -> " << tester.GetGender()
        << " -> \"\"\" << tester.genderToString() << "\"\"\" << endl;
}

// --- з) тест на валидизатора на ЕГН ---
cout << "---- Тест: валидност на ЕГН ----" << endl;
const char* arrEGN[] = {
    "111111110", // валидно (пример от условието)
    "111111104", // валидно (пример от условието)
    "111111111", // грешна контролна цифра
    "11111111",  // по-малко от 10 цифри

```

```

        "111111111a",    // съдържа буква
        "1199111110",    // несъществуващ месец (99)
        "1111321110"    // несъществуващ ден (32)
};
CPerson2 validator;
for (size_t i = 0; i < sizeof(arrEGN) / sizeof(arrEGN[0]); i++)
    cout << " " << arrEGN[i] << " -> "
           << (validator.isValid(arrEGN[i]) ? "валидно" : "НЕвалидно") << endl;

// --- и) SetEGN отхвърля невалидна стойност ---
cout << "--- Тест: SetEGN пази старата стойност ---" << endl;
CPerson2 person3("Стефан Николов", "мъж", "1111111110");
cout << " преди: " << person3.GetEGN() << endl;
bool bResult = person3.SetEGN("1234567890");           // невалидно
cout << " SetEGN(\"1234567890\") върна: " << (bResult ? "true" : "false") << endl;
cout << " след: " << person3.GetEGN() << endl;

// Задържа конзолния прозорец отворен при стартиране с F5 (debug)
getchar();
return 0;
}

```

## Очакван изход

```
--- Обикновени обекти ---
Неизвестен (неопределен), ЕГН: (няма)
Иван Петров (мъж), ЕГН: 1111111110
--- След промяна ---
Мария Иванова (жена), ЕГН: 1111111104
--- Масив от обекти ---
Георги Стоянов (мъж), ЕГН: 1111111110
Елена Димитрова (жена), ЕГН: 1111111104
Неизвестен (неопределен), ЕГН: (няма)
--- Указател (динамична памет) ---
Петър Колев (мъж), ЕГН: 1111111110
--- Референция ---
Иван Петров (жена), ЕГН: 1111111110
--- Константен обект ---
Анна Тодорова / жена / 1111111104
--- Тест: текст -> Gender -> текст ---
  "мъж" -> 0 -> "мъж"
  "жена" -> 1 -> "жена"
  "М" -> 0 -> "мъж"
  "Ж" -> 1 -> "жена"
  "male" -> 0 -> "мъж"
  "female" -> 1 -> "жена"
  "нещо" -> 2 -> "неопределен"
--- Тест: валидност на ЕГН ---
  1111111110 -> валидно
  1111111104 -> валидно
  1111111111 -> НЕвалидно
  1111111111 -> НЕвалидно
  111111111a -> НЕвалидно
  1199111110 -> НЕвалидно
  1111321110 -> НЕвалидно
--- Тест: SetEGN пази старата стойност ---
  преди: 1111111110
  SetEGN("1234567890") върна: false
  след: 1111111110
```

След последния ред програмата чака натискане на .

## Чести грешки

| Код               | Съобщение                                           | Причина и решение                                                                                           |
|-------------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| C2248             | cannot access private member                        | Опит за пряк достъп до член-променлива отвън. Използвайте <code>GetName()</code> / <code>SetName()</code> . |
| C2065             | 'string': undeclared identifier                     | Липсва <code>#include &lt;string&gt;</code> или <code>using namespace std;</code>                           |
| C2039             | 'stringToGender': is not a member of 'CPerson2'     | Функцията е дефинирана след класа, но не е <b>обявена</b> вътре в него.                                     |
| C2660             | function does not take 1 arguments                  | Пропуснат е операторът <code>CPerson2::</code> пред името при дефиниране извън класа.                       |
| C2664             | cannot convert argument 1                           | При <code>SetGender</code> с текст в кавички добавете <code>string(...)</code> около него.                  |
| C2662             | cannot convert 'this' pointer from 'const CPerson2' | Извиквате неконстантна функция върху константен обект. Ако е тестът от 8.6 — очаквано.                      |
| C4530             | C++ exception handler used                          | Липсва флагът <code>/EHsc</code> при компилация от команден ред.                                            |
| LNK2005           | main already defined                                | В проекта има и <code>struct.cpp</code> . Направете отделен проект.                                         |
| EGN не се записва |                                                     | <code>SetEGN</code> отхвърля невалидни стойности — това е замисълът. Проверете върнатата стойност.          |

## Забележка към сигнатурата на `isValid`

Функцията е написана точно както е зададена в условието: `bool isValid(const string& strEGN);` — нестатична и неконстантна член-функция. Затова не може да бъде извикана върху константен обект и изисква съществуващ обект, за да се използва.

Тъй като не чете нито една член-променлива, по-правилно от гледна точка на езика би било `static bool isValid(const string& strEGN);`. Тогава извикването става без обект: `CPerson2::isValid("111111110")`. Промяната е препоръчителна само ако преподавателят допуска отклонение от зададената сигнатура.